Authentication and Authorization Interview Notes

Short interview-focused notes for quick revision.

Related full guide:

- authentication-authorization-guide.md

---

1. Authentication vs Authorization

Authentication

- Authentication means verifying identity
- It answers: `Who are you?`
- Examples:
- username/password
- OTP
- login with Google
- biometrics

Authorization

- Authorization means deciding access rights
- It answers: `What are you allowed to do?`
- Examples:
- can access `/admin`
- can delete invoice
- can read only own profile

One-line difference

- Authentication comes first
- Authorization comes after authentication

Interview answer

```
Authentication verifies the identity of a user or client, while authorization
determines what that authenticated user or client is allowed to access.
```

---

2. Session vs Token Authentication

Session-based auth

- Server stores login state
- Browser stores session cookie
- Common in traditional web apps

Pros:

- simple for browser applications
- easy logout and session invalidation

Cons:

- server-side state required

Token-based auth

- Client stores token
- Client sends bearer token on requests
- Common in REST APIs, SPAs, mobile apps, microservices

Pros:

- stateless APIs
- works well across distributed systems

Cons:

- token storage and revocation need care

Interview answer

```
Session auth keeps authenticated state on the server and identifies the session by cookie. Token auth usually keeps the API stateless by having the client send a bearer token on each request.
```

---

3. OAuth 2.0 Basics

What is OAuth 2.0?

- OAuth 2.0 is an authorization framework
- It is mainly about delegated access
- It allows one application to access a resource on behalf of a user without sharing the user's password

Main actors

- `Resource Owner` = usually the user
- `Client` = application requesting access
- `Authorization Server` = issues tokens
- `Resource Server` = API serving protected resources

Important point

- OAuth2 is not primarily a login protocol
- OpenID Connect adds the identity layer

Interview answer

```
OAuth 2.0 is a delegated authorization framework where a client obtains tokens
from an authorization server and uses them to access protected resources on a
resource server.
```

---

4. OpenID Connect Basics

What is OIDC?

- OpenID Connect is an identity layer on top of OAuth 2.0
- It is used for authentication and SSO
- It introduces the `ID token`

Why do we need OIDC?

- OAuth2 alone does not standardize user identity for login
- OIDC standardizes authentication, identity claims, and user info

Interview answer

```
OpenID Connect extends OAuth 2.0 by adding an identity layer, mainly through
the ID token and standard user identity claims.
```

---

5. Access Token vs Refresh Token vs ID Token

Access Token

- Used to call protected APIs
- Sent to the resource server
- Represents authorization

Refresh Token

- Used to obtain a new access token
- Sent to the authorization server
- Usually long-lived compared to access token

ID Token

- Used to represent authenticated user identity
- Sent to the client application
- Used in OIDC

Common mistake

- ID token should not normally be used to call APIs

Interview answer

```
An access token is used for API access, a refresh token is used to get a new access token, and an ID token is used in OpenID Connect to carry user identity information.
```

---

6. Common OAuth2 Flows

Authorization Code Flow

- Used by web apps and modern login systems
- User authenticates at authorization server
- Client receives authorization code
- Client exchanges code for tokens

Authorization Code + PKCE

- Best for SPA, mobile, desktop, native apps

- Protects authorization code flow for public clients

Client Credentials Flow

- Used for machine-to-machine communication
- No end user involved

Refresh Token Flow

- Used to renew access tokens

Device Authorization Flow

- Used by TV apps, CLI tools, input-constrained devices

Interview answer

```
The most common modern flows are authorization code, authorization code with PKCE, client credentials, refresh token, and device authorization flow. Authorization code with PKCE is now preferred for public clients such as SPA and mobile apps.
```

---

7. PKCE

Full form

- `Proof Key for Code Exchange`

Why PKCE is used

- Public clients cannot securely keep a client secret
- PKCE protects the authorization code flow from code interception attacks

Core idea

- Client creates `code_verifier`
- Client derives `code_challenge`
- Sends challenge in auth request
- Sends verifier in token request
- Authorization server verifies they match

Best use cases

- SPA
- mobile apps
- desktop apps
- native apps

Interview answer

```
PKCE secures the authorization code flow by binding the authorization request
and token exchange together with a code challenge and code verifier. It is
especially important for public clients like mobile and SPA applications.
```

---

8. Auth Server vs Resource Server

Authorization Server

- Authenticates users or clients
- Issues tokens
- May expose authorization, token, revocation, introspection, and JWK endpoints

Resource Server

- Hosts protected APIs/resources
- Validates access tokens
- Grants or denies access

Interview answer

```
The authorization server issues tokens and handles authentication, while the
resource server validates access tokens and protects APIs or other resources.
```

---

9. RBAC

Full form

- `Role-Based Access Control`

Concept

- Users are assigned roles
- Roles are assigned permissions
- Access decisions are based on roles and permissions

Example

- `ADMIN` -> create user, delete user, read reports
- `USER` -> read own profile
- `AUDITOR` -> read reports only

Why RBAC is useful

- easier to manage
- maps to organization structure
- reduces repeated permission assignments

Interview answer

```
RBAC is an authorization model where permissions are assigned to roles and
users are assigned those roles, making access management more scalable and
easier to administer.
```

---

10. Spring Security Mapping

Common Spring features

- `formLogin()` -> browser login with session
- `httpBasic()` -> HTTP Basic authentication
- `oauth2ResourceServer().jwt()` -> bearer JWT API protection
- `oauth2Login()` -> login with OAuth2/OIDC providers
- `@EnableMethodSecurity` -> method-level authorization
- `@PreAuthorize(...)` -> secure specific methods

Roles vs authorities in Spring

- `hasRole("ADMIN")` usually checks `ROLE_ADMIN`
- `hasAuthority("invoice:read")` checks exact authority string

Interview answer

```
In Spring Security, authentication establishes the principal, and authorization
is enforced through request matchers or method security using roles and
authorities.
```

---

11. Spring Authorization Server

What it is

- Official Spring project for building an OAuth2 Authorization Server and OpenID Connect provider

Use cases

- central login system
- token issuance
- SSO for multiple applications
- custom identity platform in Spring

Interview answer

```
Spring Authorization Server is the official Spring project used to build an
authorization server and OIDC identity provider that can issue tokens and
support centralized authentication.
```

---

12. Frequently Asked Interview Questions

Q1. What is the difference between authentication and authorization?

```
Authentication verifies identity, while authorization decides permissions after
identity is verified.
```

Q2. What is OAuth 2.0?

```
OAuth 2.0 is a delegated authorization framework that lets a client obtain
tokens to access protected resources.
```

Q3. What is OpenID Connect?

```
OpenID Connect is an identity layer on top of OAuth 2.0 that standardizes user authentication and identity claims.
```

Q4. What is the difference between access token and refresh token?

```
An access token is used to access APIs, while a refresh token is used to obtain a new access token after the old one expires.
```

Q5. What is an ID token?

```
An ID token is an OIDC token that contains identity information about the authenticated user and is intended for the client application.
```

Q6. What is PKCE and why is it important?

```
PKCE secures the authorization code flow by preventing intercepted authorization codes from being exchanged by attackers, especially in public clients.
```

Q7. What is RBAC?

```
RBAC is Role-Based Access Control, where access is managed by assigning permissions to roles and roles to users.
```

Q8. What is the difference between authorization server and resource server?

```
The authorization server authenticates and issues tokens, while the resource server validates tokens and protects resources.
```

Q9. Why should passwords not be stored in plain text?

```
Passwords must be stored as secure hashes so that a database leak does not directly expose user credentials.
```

Q10. What is the difference between role and authority in Spring Security?

```
A role is usually a high-level business grouping like ADMIN, while an authority is a specific permission string. In Spring, roles are commonly represented internally with the ROLE_ prefix.
```

---

13. Very Short Revision Sheet

- Authentication = who are you
- Authorization = what can you do
- Session = server-side login state
- Access token = used to call API
- Refresh token = used to get new access token
- ID token = used to identify authenticated user in OIDC
- OAuth2 = delegated authorization
- OIDC = identity layer on top of OAuth2
- PKCE = secures auth code flow for public clients
- RBAC = roles organize permissions
- Authorization Server = issues tokens
- Resource Server = protects APIs
- Spring Security = secures apps
- Spring Authorization Server = builds auth server in Spring

---

14. Official References

- Spring Security reference: https://docs.spring.io/spring-security/reference/index.html
- Spring Authorization Server reference: https://docs.spring.io/spring-authorization-server/reference/index.html
- OAuth 2.0 RFC 6749: https://www.rfc-editor.org/rfc/rfc6749
- Bearer Token RFC 6750: https://www.rfc-editor.org/rfc/rfc6750
- PKCE RFC 7636: https://www.rfc-editor.org/rfc/rfc7636
- OAuth 2.0 for Native Apps RFC 8252: https://www.rfc-editor.org/rfc/rfc8252
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html
- NIST RBAC project: https://csrc.nist.gov/projects/role-based-access-control